PRACTICING WEEK 5

Course: Data Engineering

Name: Meng Oudom

Goal: Last week you have some data for online store. You already ingested API data into OLTP tables: customers, products, orders, order_items. Now you must **serve** the data to 3 consumers:

1. **BI team** (dashboards, reports)
2. **Ops team** (near real-time alerts)
3. **Sales team** (needs scores inside CRM → Reverse ETL)

I. Build a small data mart (star-style) using SQL views

Create 3 views in your warehouse layer (or pretend schema mart):

a) mart.dim_customer

- customer_id, name, phone

customer_id	name	phone (City)
1	Sok Dara	Phnom Penh
2	Kim Sreymom	Battambang
3	Chanthy	Siem Reap

b) mart.dim_product

- product_id, name, category, price

product_id	name	category	price
10	Banana	Fresh fruit	0.50
11	Apple	Fresh fruit	0.80
12	Candy	Candy	0.30
13	Milk	Dairy	1.20

c) mart.fact_sales (grain: order_item)

- order_id, order_date, store, customer_id, product_id, quantity, price, revenue
- where revenue = quantity * price

order_item_id	order_id	order_date	store	customer_id	product_id	quantity	price	revenue
1	1001	2026-01-02	Store_A	1	10	10	0.50	5.00
2	1001	2026-01-02	Store_A	1	12	5	0.30	1.50
3	1002	2026-01-03	Store_A	1	11	3	0.80	2.40
4	1003	2026-01-03	Store_B	2	10	20	0.50	10.00
5	1003	2026-01-03	Store_B	2	13	2	1.20	2.40
6	1004	2026-01-04	Store_A	3	12	10	0.30	3.00

⇨ **Rules**

- JOIN: orders + order_items + products
- Ensure quantity > 0 and price >= 0 (filter bad rows)

II. Serve for Analytics using File Exchange

Write one SQL query that outputs a CSV-style result:

Columns:

- month (YYYY-MM)
- store
- total_orders
- total_revenue

month	store	total_orders	total_revenue
2026-01	Store_A	3	11.90
2026-01	Store_B	1	12.40

Then answer:

- *When is file exchange a good idea?*
  + It is a good idea for occasional sharing (e.g., weekly reports) when the consumer doesn't have technical access to the database or when you need to share a static snapshot of data with external partners who use tools like Excel or Google Sheets.
- *What is the biggest limitation if you email Excel files?*
  + The biggest limitation is the loss of the "Single Source of Truth." Once an Excel file is emailed, it becomes a static, disconnected copy. It's difficult to keep the data fresh, and different versions of the same report often circulate among teams, leading to data inconsistency and confusion.

III. Query Federation

Assume:

- Warehouse has mart.fact_sales
- A Product API provides the latest product category changes (external source)

Write:

- *A short explanation: why federation is useful for exploration*

  Query federation allows a user to query data across multiple disparate systems (e.g., a SQL warehouse and an external product API) as if they were in a single database, without having to first ingest and move all that data into a central storage layer.

  It is extremely useful for exploration because:

  - **Speed to Insight**: Data engineers don't need to build complex ETL pipelines just to "peek" at data or join a small amount of external data with internal facts.
  - **Access to Live Data**: It allows joining historical warehouse data with "live" or "hot" data from external APIs or operational databases that hasn't been synced yet.
  - **Reduced Storage Costs**: You don't pay to store data that you only need to query occasionally.
- *One risk (production overload) + one control (limit, caching, read replica, etc.)*
  - **One Risk: Production Overload**

    Federated queries often translate into direct calls against the source systems' resources (CPU, Memory, IO). A complex join involving a large warehouse table and a production transactional database (or API) could significantly slow down or даже crash the production system by consuming its resources or locking tables.

- **One Control: Read Replicas / Caching**
  To mitigate this risk, query federation should ideally target a **Read Replica** of the production database instead of the primary instance. Additionally, implement **Caching** layers or query limits (timeouts and row limits) to ensure that expensive cross-source queries don't run indefinitely and degrade source performance.

IV. Reverse ETL

Sales team works in CRM, not in warehouse. Reverse ETL sends processed/scored data back to operational tools.

1. *Create a table crm_customer_scores(customer_id, spend_score, updated_at) OR add a column in customers (if allowed).*
2. *Compute spend_score = total_spend from vw_order_enriched (customer-level aggregation).*
3. *Insert/update scores into crm_customer_scores.*

⇨ **Deliverable:** *SQL 04_reverse_etl.sql + screenshot/result table.*

customer_id	spend_score	updated_at
2	12.40	2026-02-11 15:51:53
1	8.90	2026-02-11 15:51:53
3	3.00	2026-02-11 15:51:53

Homework
(Work as groups)

I. Streaming Systems (near real-time)

Serving data for operational analytics (real-time dashboards/alerts)
Requirements: Design 1 event + 1 alert using Kafka + Python consumer

1. Define an event schema (JSON): order_created

Your order_created event must include these fields:

- event_id
- event_time
- order_id
- customer_id
- store
- total_items
- total_amount

2. Create an alert rule

Trigger an alert when:

- total_amount > 500 OR
- total_items > 20

⇨ **Tools (Kafka + Python Consumer)**

Use the following tools:

- Docker Desktop
- Kafka via docker-compose
- Python consumer using:
  - confluent-kafka or
  - kafka-python
- Kafka topic**:** `orders`

II. Build Machine Learning Model (Linear Regression: Predict order_total)

- Target is continuous (fits regression).
- Uses your real tables from last week (orders + items + products + store). Practicing

⇨ **What do you need to do?**

1. Build ML dataset (one row per order) with features:
    - total_qty
    - item_lines (count of order_items)
    - avg_price, max_price
    - store (categorical → one-hot)
    - day_of_week (from order_date)
    - Target: order_total
2. Train/test split (80/20)
3. Train **Multiple Linear Regression**
4. Evaluate: MAE, RMSE, R^2
5. Explain coefficients: which features increase order_total?
6. Save predictions as order_predictions.csv

Deliverables

- homework_ml.ipynb (or .py)
- order_predictions.csv